SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Design Task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Design Task

1. **Hybrid AI Recommendation Engine Design**
   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
   Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
   Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
   Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
   Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

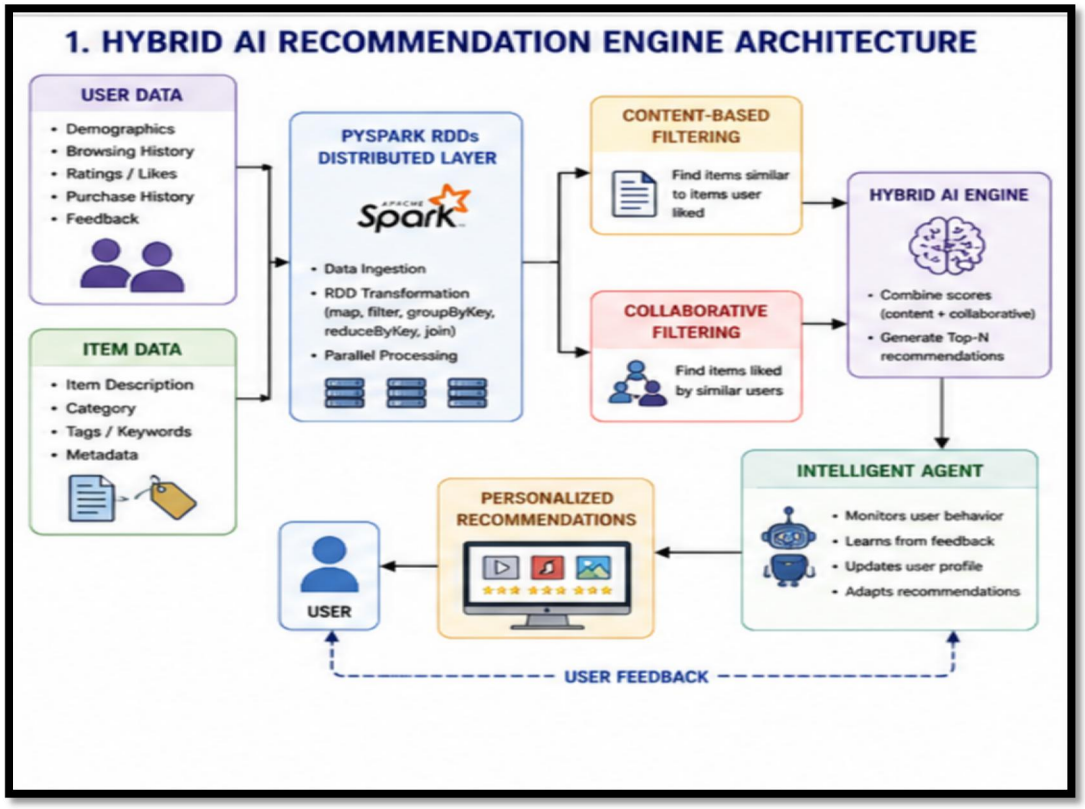
1. Hybrid AI Recommendation Engine Design:

Introduction

A Hybrid AI Recommendation Engine combines content-based filtering and collaborative filtering to provide accurate and personalized recommendations. The system uses PySpark RDDs to process a large number of user-item interactions in a distributed manner.

For example, an online shopping or movie platform may have millions of users and products/movies. Processing all this information on a single machine can be slow. PySpark distributes the data and computations across multiple worker nodes.

System Architecture:



1. Data Collection

The system collects:

- User ID
- Item ID
- Ratings
- Likes/dislikes
- Purchase history
- Search history
- Browsing history

- Item category
- Item description
- Keywords or tags

For example:

User	Item	Rating
U1	Movie A	5
U1	Movie B	4
U2	Movie A	5
U2	Movie C	4

These records can be converted into PySpark RDDs.

2. Content-Based Filtering

Content-based filtering recommends items based on the characteristics of items previously liked by the user.

For example, if a user frequently watches:

- Action
- Science fiction
- Adventure

the system searches for other items with similar features.

The advantage is that recommendations can be generated based on the user's own preferences.

3. Collaborative Filtering

Collaborative filtering uses the behavior of multiple users.

If User A and User B have similar interests, items liked by User B can be recommended to User A.

For example:

User A → Movie A, Movie B

User B → Movie A, Movie B, Movie C

Since A and B have similar preferences, Movie C can be recommended to User A.

4. Hybrid Recommendation

The system combines both approaches.

For example:

Final Score = 0.4 × Content Score + 0.6 × Collaborative Score

The weights can be changed according to system performance.

This approach reduces the limitations of using only one recommendation technique.

5. Distributed Processing Using PySpark RDDs

PySpark RDDs divide large datasets into partitions.

Important RDD operations include:

- map() – transforms data.
- filter() – removes unwanted records.
- groupByKey() – groups records.
- reduceByKey() – combines values.
- join() – combines related datasets.

For millions of users and products, different partitions can be processed simultaneously on different worker nodes.

Therefore:

Large Dataset → Partitions → Parallel Processing → Faster Recommendation

6. Role of Intelligent Agents

An intelligent recommendation agent continuously observes user behavior.

The agent can:

1. Monitor user clicks.
2. Monitor ratings.
3. Identify frequently viewed categories.
4. Detect skipped recommendations.
5. Update recommendation weights.
6. Generate new recommendations.

For example, if a user starts watching many AI-related videos, the agent detects the new interest and increases the priority of AI content.

7. Feedback Mechanism

User feedback is returned to the intelligent agent.

Recommendation

↓

User Interaction

↓

Feedback

↓

Agent Analysis

↓

Updated Preferences

↓

New Recommendation

Advantages

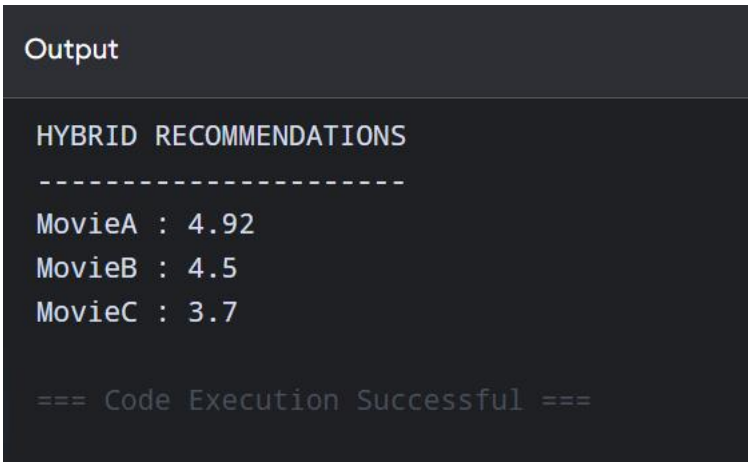
- Highly personalized.
- Handles millions of interactions.
- Scalable using PySpark.
- Combines two recommendation methods.
- Adapts to changing user interests.
- Reduces recommendation errors.

PYTHON CODE :

```
data = [
    ("U1", "MovieA", 5),
    ("U1", "MovieB", 4),
    ("U2", "MovieA", 5),
    ("U2", "MovieC", 4),
    ("U3", "MovieB", 5),
    ("U3", "MovieC", 3)
]
item_ratings = {}
for user, item, rating in data:
    if item not in item_ratings:
        item_ratings[item] = []
    item_ratings[item].append(rating)
item_average = {}
for item, ratings in item_ratings.items():
    item_average[item] = sum(ratings) / len(ratings)
content_scores = {
    "MovieA": 4.8,
```

```
    "MovieB": 4.5,
    "MovieC": 4.0
}
final_scores = {}
for item in content_scores:
    content = content_scores[item]
    collaborative = item_average[item]
    score = (0.4 * content) + (0.6 * collaborative)
    final_scores[item] = score
recommendations = sorted(
    final_scores.items(),
    key=lambda x: x[1],
    reverse=True
)
print("HYBRID RECOMMENDATIONS")
print("----------------------")
for item, score in recommendations:
    print(item, ":", round(score, 2))
```

OUTPUT :



2. Smart Disaster Detection System :

Introduction

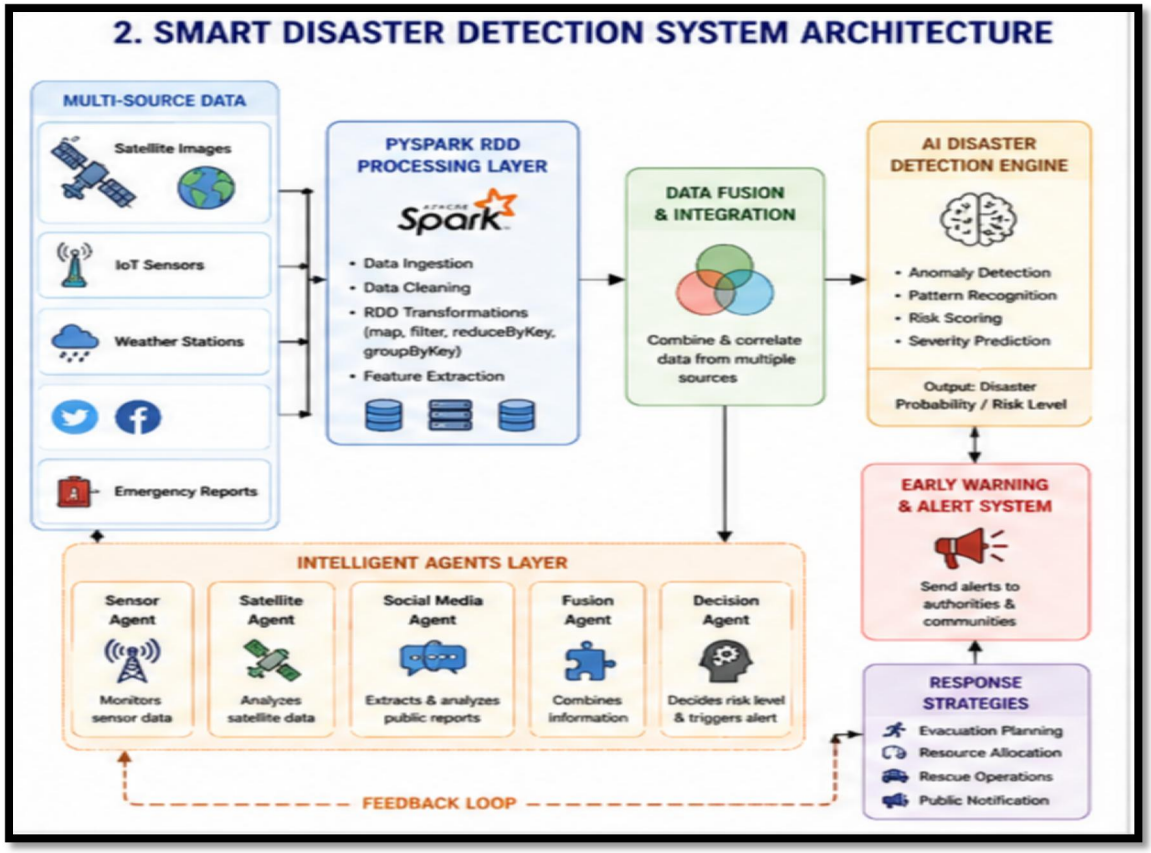
A Smart Disaster Detection System uses AI and distributed processing to detect natural disasters such as floods, earthquakes, cyclones and landslides.

The system collects information from:

- Satellite images
- IoT sensors
- Weather stations
- Seismic sensors
- Social media
- Emergency reports

PySpark processes these large-scale data sources, while intelligent agents coordinate detection and warning activities.

System Architecture:



1. Satellite Data

Satellite information can provide:

- Cloud patterns
- Flooded areas
- Temperature

- Forest fires
- Cyclone formation
- Changes in land conditions

Satellite images can be processed to identify abnormal geographical patterns.

2. Sensor Data

Sensors provide real-time measurements such as:

- Rainfall
- Water level
- Temperature
- Wind speed
- Atmospheric pressure
- Seismic activity

For example, a sudden increase in river water level combined with heavy rainfall may indicate a possible flood.

3. Social Media Data

Social media can provide real-time information from people in affected areas.

The system can identify:

- Disaster-related keywords
- Location information
- Emergency reports
- Images
- Videos

For example:

"River water is rising rapidly near the village."

"Heavy flooding in the area."

"Road completely covered by water."

These reports can provide additional evidence.

4. PySpark RDD Processing

Each source can be represented as an RDD.

For example:

Sensor RDD

Satellite RDD

Social Media RDD

RDD transformations such as map(), filter(), reduceByKey() and join() can process these datasets in parallel.

5. Data Fusion

Data fusion means combining information from multiple sources.

For example:

Heavy Rainfall → High Risk

High Water Level → High Risk

Satellite Flood Area → High Risk

Social Media Reports → High Risk

When all these sources indicate the same event, the confidence of disaster detection increases.

6. AI Disaster Detection

The AI system can use:

- Anomaly detection
- Classification
- Pattern recognition
- Image analysis
- Risk scoring

The output may be:

Flood Probability = 85%

Risk Level = High

7. Intelligent Agents : Different intelligent agents can perform different tasks.

- **Sensor Agent :** Monitors real-time sensor information.
- **Satellite Agent :** Analyzes satellite images and geographical changes.
- **Social Media Agent :** Analyzes public reports and disaster-related posts.
- **Fusion Agent :** Combines information from all agents.
- **Decision Agent :** Determines the severity and whether a warning should be issued.

8. Early Warning

If the calculated risk exceeds a threshold, the system generates an alert.

For example:

Flood Risk: 85%

Status: HIGH RISK

Action: Issue Early Warning

The warning can be sent to authorities and emergency response systems.

9. Response Strategies

The system can support:

- Evacuation planning
- Emergency resource allocation
- Ambulance routing
- Rescue team deployment
- Public notifications
- Authority notifications

Advantages

- Real-time disaster detection.
- Combines multiple data sources.
- Distributed processing.
- Faster warning generation.
- Intelligent coordination.
- Can process very large datasets.

PYTHON CODE :

```
data = [
    ("Sensor", "Rainfall", 95),
    ("Sensor", "WaterLevel", 90),
    ("Satellite", "FloodArea", 80),
    ("SocialMedia", "FloodReports", 75),
    ("Sensor", "Temperature", 30)
]
high_risk = []
for record in data:
    source, event, value = record
    if value >= 70:
        high_risk.append(record)
print("HIGH RISK DATA")
print("--------------")
```

```
for record in high_risk:
    print(record)
source_count = {}
for record in high_risk:
    source = record[0]
    if source not in source_count:
        source_count[source] = 0
    source_count[source] += 1
print("\nSOURCE SUMMARY")
print("--------------")
for source, count in source_count.items():
    print(source, ":", count)
total = 0
for record in high_risk:
    total += record[2]
risk_score = total / len(high_risk)
print("\nDISASTER RISK SCORE:", round(risk_score, 2))
if risk_score >= 70:
    print("WARNING: POSSIBLE FLOOD DETECTED")
else:
    print("STATUS: NO MAJOR DISASTER")
```

OUTPUT :

```
Output

HIGH RISK DATA
--------------
('Sensor', 'Rainfall', 95)
('Sensor', 'WaterLevel', 90)
('Satellite', 'FloodArea', 80)
('SocialMedia', 'FloodReports', 75)

SOURCE SUMMARY
--------------
Sensor : 2
Satellite : 1
SocialMedia : 1

DISASTER RISK SCORE: 85.0
WARNING: POSSIBLE FLOOD DETECTED

=== Code Execution Successful ===
```

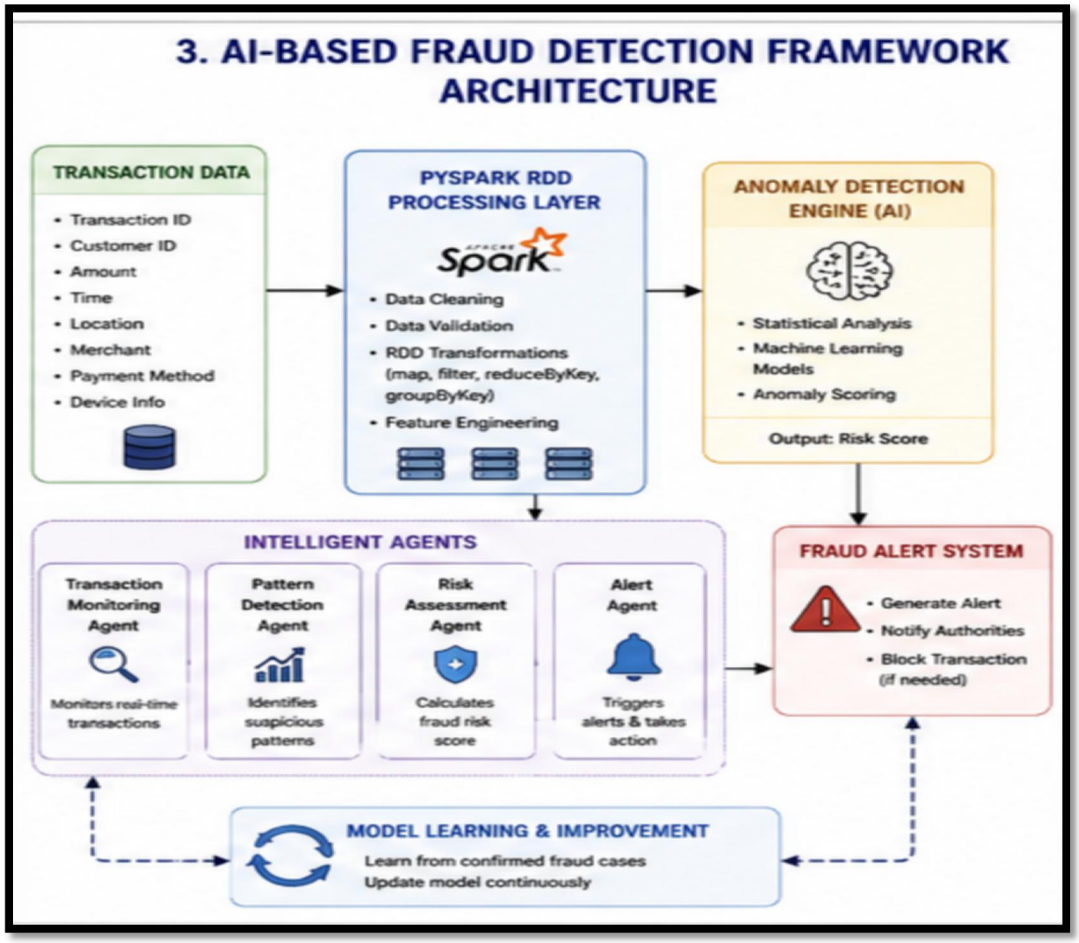
3. AI-Based Fraud Detection Framework :

Introduction

Financial institutions process millions of transactions every day. Manually checking every transaction is impossible.

An AI-Based Fraud Detection Framework uses PySpark RDDs and anomaly detection to identify suspicious transactions.

System Architecture:



1. Transaction Data

The system may process:

- Transaction ID
- Customer ID
- Transaction amount
- Time
- Location
- Payment method
- Merchant
- Device information

Example:

T1 C1 500

T2 C1 700

T3 C1 600

T4 C2 200

T5 C2 5000

2. Data Preprocessing

Before detecting fraud, the system removes:

- Invalid transactions
- Missing values
- Duplicate transactions
- Incorrect formats

RDD operations can perform these tasks in parallel.

For example:

rdd.filter(lambda x: x[2] > 0)

removes transactions with invalid negative or zero amounts.

3. Feature Extraction

Important features can include:

- Transaction amount
- Number of transactions
- Transaction frequency
- Location changes
- Unusual transaction time
- Average spending

These features are used to determine whether a transaction is normal or suspicious.

4. Anomaly Detection

Anomaly detection identifies transactions that significantly differ from normal behavior.

For example, suppose a customer normally spends:

₹500

₹700

₹600

₹800

and suddenly performs a transaction of:

₹50,000

The transaction may be suspicious.

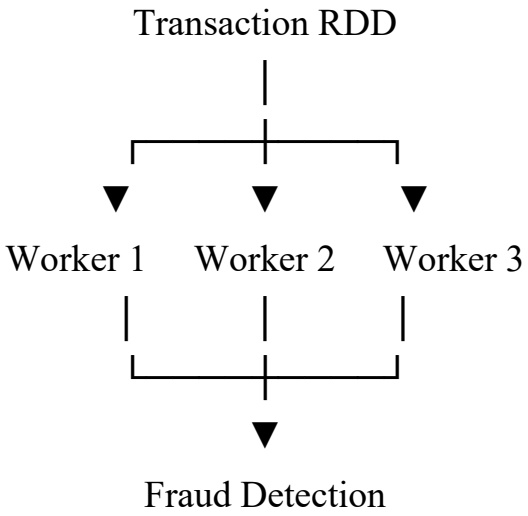
More advanced implementations can use:

- Isolation Forest
- Clustering

- Statistical anomaly detection
- Decision trees
- Random forests

5. Distributed Processing

Financial transactions are distributed among worker nodes.



This makes the system suitable for large-scale financial datasets.

6. Intelligent Agents

Transaction Monitoring Agent

Continuously monitors new transactions.

Pattern Detection Agent

Identifies unusual transaction patterns.

Risk Assessment Agent

Calculates the risk score.

Alert Agent

Generates an alert when suspicious behavior is identified.

7. Alert Generation

Example:

Transaction: T105

Amount: ₹50,000

Risk Score: High

Status: Suspicious

Action: Fraud Alert Generated

8. Continuous Improvement

When a bank confirms that a transaction was actually fraudulent, that information can be stored as historical training data.

The AI model can periodically be retrained.

New Transaction

↓

Detection

↓

Fraud Investigation

↓

Confirmed Fraud?

↓

Add to Training Data

↓

Retrain Model

Advantages

- Processes millions of transactions.
- Detects suspicious patterns quickly.
- Reduces manual monitoring.
- Supports real-time alerts.
- Continuously improves detection.
- Scalable using PySpark.

PYTHON CODE :

```
transactions = [
    ("T1", "C1", 500),
    ("T2", "C1", 700),
    ("T3", "C1", 600),
    ("T4", "C2", 200),
    ("T5", "C2", 250),
    ("T6", "C2", 5000)
]
print("FRAUD DETECTION SYSTEM")
```

```
print("----------------------")
total = 0
for transaction in transactions:
    total += transaction[2]
average = total / len(transactions)
print("Average Transaction Amount:", round(average, 2))
threshold = average * 2
suspicious = []
for transaction in transactions:
    transaction_id, customer_id, amount = transaction
    if amount > threshold:
        suspicious.append(transaction)
print("\nSUSPICIOUS TRANSACTIONS")
print("----------------------")
if len(suspicious) == 0:
    print("No suspicious transactions found.")
else:
    for transaction in suspicious:
        print(transaction)
    print("\nFRAUD ALERT: Suspicious transaction detected!")
```

OUTPUT :

```
Output

FRAUD DETECTION SYSTEM
----------------------
Average Transaction Amount: 1208.33

SUSPICIOUS TRANSACTIONS
-----------------------
('T6', 'C2', 5000)

FRAUD ALERT: Suspicious transaction detected!

=== Code Execution Successful ===
```

4. Autonomous Supply Chain Optimization System :

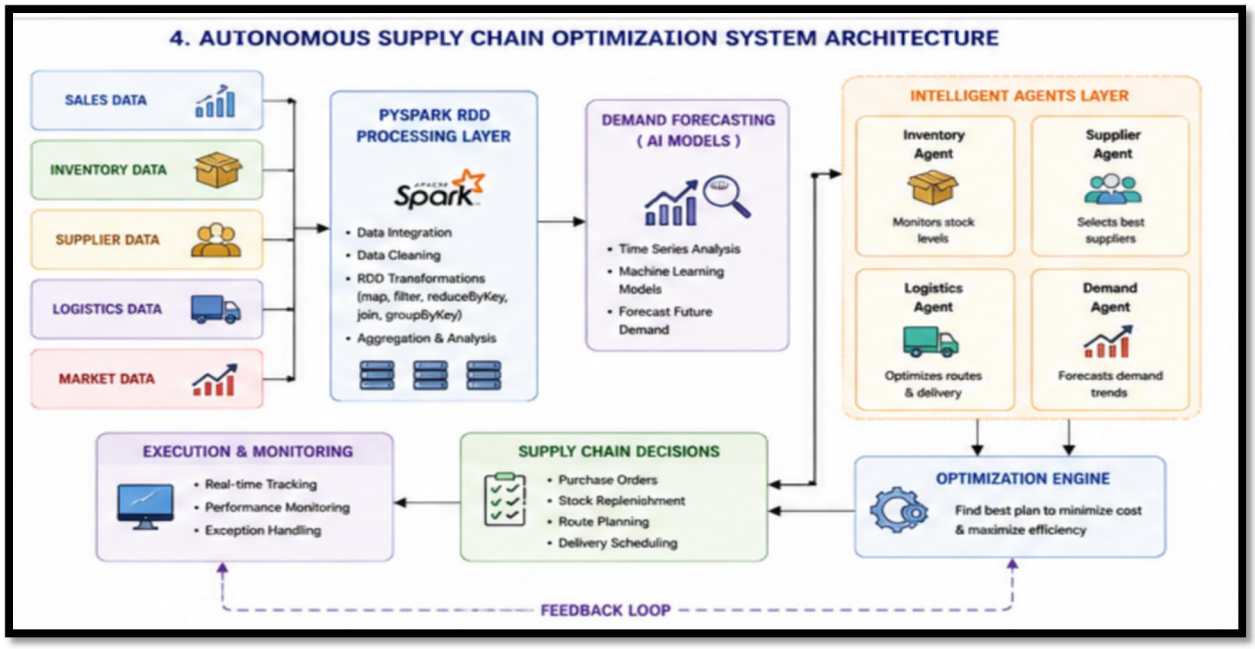
Introduction

A supply chain contains many activities such as:

- Purchasing
- Inventory management
- Demand forecasting
- Transportation
- Warehousing
- Supplier selection
- Delivery

An Autonomous Supply Chain Optimization System uses PySpark and intelligent agents to make decentralized decisions and reduce operational costs.

System Architecture:



1. Data Collection

The system collects:

Sales Data

- Product
- Quantity sold
- Date
- Location

Inventory Data

- Current stock
- Minimum stock
- Warehouse capacity

Supplier Data

- Supplier
- Price
- Delivery time
- Reliability

Logistics Data

- Distance
- Transportation cost
- Delivery time

2. PySpark RDD Processing

Suppose sales data is:

Laptop 10

Laptop 15

Phone 20

Phone 25

Tablet 8

Tablet 12

reduceByKey() can calculate total demand:

Laptop → 25

Phone → 45

Tablet → 20

This is efficient for large datasets because aggregation can be performed in a distributed manner.

3. Demand Forecasting

The system analyzes historical sales to estimate future demand.

For example:

Previous demand → 20

Current demand → 30

Expected demand → 35

The AI model can forecast future product requirements.

4. Inventory Optimization

The system compares demand with available inventory.

For example:

Product	Demand	Stock
Laptop	25	15
Phone	45	30
Tablet	20	10

Since demand is greater than stock, the system recommends reordering.

5. Intelligent Agents

- **Inventory Agent :** Monitors inventory levels.
- **Demand Agent :** Forecasts future demand.
- **Logistics Agent :** Selects efficient transportation routes.

- **Supplier Agent :** Selects suitable suppliers based on price, quality and delivery time.

6. Decentralized Decision Making

Each agent performs its own task, but the agents communicate with one another.

For example:

Demand Agent

↓

Expected demand = 100 units

↓

Inventory Agent

↓

Current stock = 40 units

↓

Supplier Agent

↓

Order 60 units

↓

Logistics Agent

↓

Select low-cost delivery route

7. Cost Minimization

The system attempts to minimize:

- Inventory cost
- Transportation cost
- Warehouse cost
- Supplier cost
- Delay cost

while maintaining adequate inventory.

8. Advantages

- Reduces inventory shortages.
- Reduces unnecessary stock.
- Improves demand forecasting.
- Reduces transportation cost.

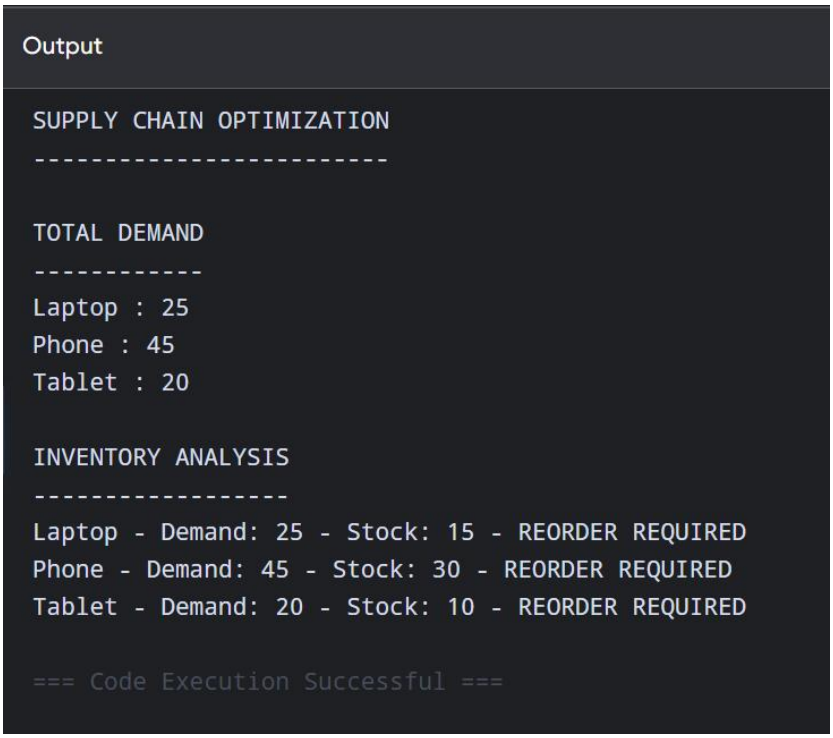
- Supports automatic decisions.
- Handles large-scale supply chain data.
- Improves overall efficiency.

PYTHON CODE :

```
sales = [
    ("Laptop", 10),
    ("Laptop", 15),
    ("Phone", 20),
    ("Phone", 25),
    ("Tablet", 8),
    ("Tablet", 12)
]
inventory = [
    ("Laptop", 15),
    ("Phone", 30),
    ("Tablet", 10)
]
demand = {}
for product, quantity in sales:
    if product not in demand:
        demand[product] = 0
    demand[product] += quantity
print("SUPPLY CHAIN OPTIMIZATION")
print("-------------------------")
print("\nTOTAL DEMAND")
print("------------")
for product, quantity in demand.items():
    print(product, ":", quantity)
stock = {}
for product, quantity in inventory:
```

```
    stock[product] = quantity
print("\nINVENTORY ANALYSIS")
print("------------------")
for product in demand:
    demand_value = demand[product]
    stock_value = stock[product]
    if demand_value > stock_value:
        status = "REORDER REQUIRED"
    else:
        status = "STOCK AVAILABLE"
    print(
        product,
        "- Demand:", demand_value,
        "- Stock:", stock_value,
        "-", status
    )
```

OUTPUT :



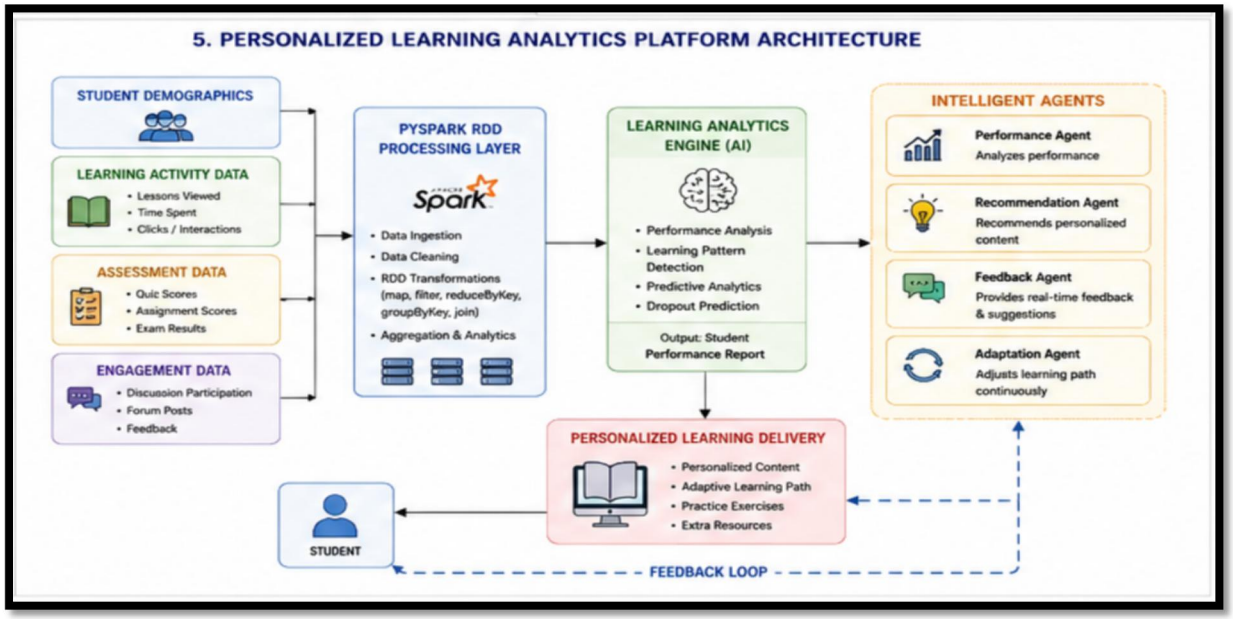
5. Personalized Learning Analytics Platform :

Introduction

A Personalized Learning Analytics Platform uses AI to analyze student learning behavior and provide customized educational content.

A university may have thousands of students. Processing all student activity on one machine can become difficult. PySpark provides scalable distributed processing.

System Architecture:



1. Data Collection

The platform collects:

- Student ID
- Subject
- Quiz score
- Assignment score
- Study time
- Number of attempts
- Lessons completed
- Questions answered
- Learning history

Example:

S1 Python 85

S1 AI 90

S2 Python 45

S2 AI 55

S3 Python 70

S3 AI 75

2. PySpark RDD Processing

Student records are converted into RDDs.

For example:

```
student_scores = rdd.map(
    lambda x: (x[0], x[2])
)
```

This creates:

S1 → 85

S1 → 90

S2 → 45

S2 → 55

S3 → 70

S3 → 75

Then groupByKey() can group scores for each student.

3. Performance Analysis

The system calculates the average performance.

Example:

S1 = (85 + 90) / 2 = 87.5

S2 = (45 + 55) / 2 = 50

S3 = (70 + 75) / 2 = 72.5

4. Adaptive Learning

The platform can categorize learners.

High Performance

Average score ≥ 80

Recommended:

- Advanced lessons
- Difficult problems
- Additional projects

Medium Performance

Average score between 60 and 79

Recommended:

- Practice exercises
- Revision materials
- Moderate-level problems

Low Performance

Average score < 60

Recommended:

- Basic lessons
- Simple examples
- Additional practice
- Revision videos

5. Intelligent Learning Agents

- **Learning Agent :** Monitors student learning activity.
- **Performance Agent :** Analyzes test and assignment scores.
- **Recommendation Agent :** Selects appropriate learning materials.
- **Feedback Agent :** Provides immediate feedback.

6. Real-Time Feedback

Suppose a student attempts an AI quiz:

Score = 40%

The system identifies that the student has difficulty with the topic.

The intelligent agent may recommend:

1. Review basic concept
2. Watch explanation video
3. Solve 5 practice questions
4. Attempt quiz again

If the student later scores 85%, the system can move the student to advanced content.

7. Continuous Adaptation

The system follows:

Student Activity

↓

Data Collection

↓

Performance Analysis

↓

Recommendation

↓

Student Attempts Content

↓

Feedback

↓

Agent Updates Student Profile

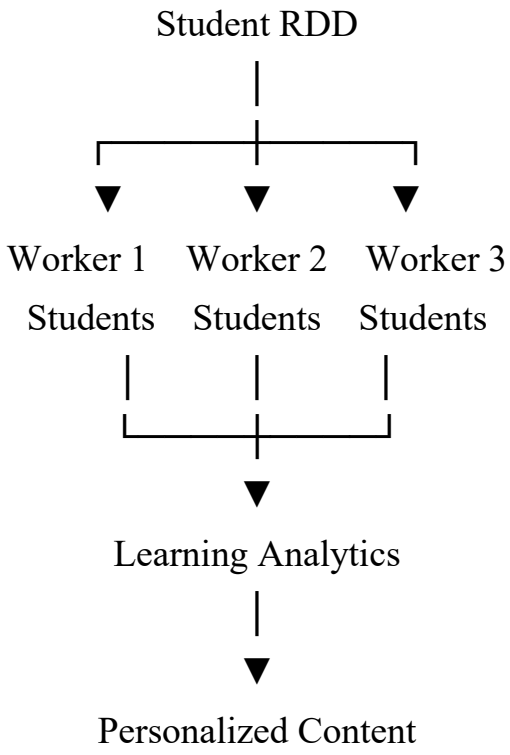
↓

New Recommendation

Therefore, the platform continuously adapts to each student's learning progress.

8. Scalability

For thousands of students, the data can be divided into partitions.



Advantages

- Personalized learning.
- Supports thousands of learners.
- Real-time feedback.

- Detects weak subjects.
- Recommends suitable content.
- Reduces learning gaps.
- Scalable using PySpark.
- Continuously adapts to student performance.

PYTHON CODE :

```
students = [
    ("S1", "Python", 85),
    ("S1", "AI", 90),
    ("S2", "Python", 45),
    ("S2", "AI", 55),
    ("S3", "Python", 70),
    ("S3", "AI", 75)
]
student_scores = {}
for student, subject, score in students:
    if student not in student_scores:
        student_scores[student] = []
    student_scores[student].append(score)
print("PERSONALIZED LEARNING ANALYTICS")
print("--------------------------------")
for student in sorted(student_scores):
    scores = student_scores[student]
    average = sum(scores) / len(scores)
    if average >= 80:
        recommendation = "Advanced Learning"
    elif average >= 60:
        recommendation = "Practice Exercises"
    else:
        recommendation = "Basic Lessons + Extra Practice"
```

```
    print(
        student,
        "- Average:", round(average, 2),
        "- Recommendation:", recommendation
    )
```

OUTPUT :